



Test technique – Développeur Full Stack Senior (React / Node.js)

Durée estimée : 4 heures maximum

Stack : React, Node.js, TypeScript (optionnel mais apprécié)

Contexte

Vous devez développer un système de gestion de contenus éditoriaux avec back-office d'administration, système de notifications et gestion avancée des utilisateurs. L'application simule une plateforme de publication avec segmentation par réseau d'utilisateurs.

Ce test reflète les problématiques réelles rencontrées sur nos projets : intégration de données externes, back-office complexe, gestion de workflows et génération de contenus.

Objectifs

Créer une application Full Stack comprenant :

Back-end (Node.js)

- API REST complète pour articles, catégories et utilisateurs
- Système de segmentation par réseau utilisateur
- Import et structuration de données depuis un fichier JSON (simulation CMS)
- Système de notifications email (génération HTML)
- Gestion avancée des permissions (admin vs éditeur)
- Validation robuste des données

Front-end (React)

- Back-office administrateur avec interface professionnelle
- Dashboard avec statistiques et dernières publications
- Gestion CRUD complète des articles avec prévisualisation



- Système de filtrage multi-critères avancé
- Gestion des catégories et des réseaux utilisateurs
- Module de notifications (historique des emails envoyés)
- Gestion des états de chargement et erreurs



Spécifications fonctionnelles

Entités principales

Article

```
{
  id: string,
  title: string,
  content: string,
  excerpt: string,
  author: string,
  categories: string[], // IDs des catégories
  network: string, // ID du réseau utilisateur
  status: 'draft' | 'published' | 'archived',
  featured: boolean,
  publishedAt: Date | null,
  createdAt: Date,
  updatedAt: Date
}
```

Category

```
{
  id: string,
  name: string,
  slug: string,
  description: string,
  color: string // code couleur hex
}
```

Network

```
{
  id: string,
  name: string,
  description: string
}
```

EmailNotification

```
{
  id: string,
  articleId: string,
  recipients: string[], // emails
  subject: string,
  sentAt: Date,
  status: 'sent' | 'failed'
}
```



Endpoints API attendus

Articles

GET /api/articles - Liste paginée avec filtres avancés
GET /api/articles/:id - Détail
POST /api/articles - Création
PUT /api/articles/:id - Modification
DELETE /api/articles/:id - Suppression
PATCH /api/articles/:id/status - Changer le statut
POST /api/articles/:id/notify - Envoyer notification email

Categories

GET /api/categories - Liste complète
POST /api/categories - Création
PUT /api/categories/:id - Modification
DELETE /api/categories/:id - Suppression

Networks

GET /api/networks - Liste complète

Data Import

POST /api/import/articles - Importer articles depuis JSON

Notifications

GET /api/notifications - Historique des notifications

Fonctionnalités Front-end obligatoires

1. Dashboard

- Statistiques : nombre total d'articles, par statut, par réseau
- Graphique simple (barres ou camembert) de répartition par catégorie
- Liste des 5 derniers articles publiés
- Dernières notifications envoyées

2. Gestion des articles

- Tableau avec tri sur colonnes (titre, date, statut, réseau)
- Recherche en temps réel sur titre et contenu
- Filtres multiples combinables :
 - Par statut (draft, published, archived)
 - Par catégorie (multi-sélection)
 - Par réseau
 - Articles mis en avant uniquement
- Pagination (20 items par page)
- Actions rapides : éditer, supprimer, archiver, mettre en avant
- Changement de statut en masse (sélection multiple)

3. Formulaire d'édition/création d'article

- Tous les champs de l'entité Article
- Éditeur de texte enrichi (simple textarea acceptable, mais éditeur riche apprécié)
- Sélection multiple des catégories avec badges
- Toggle "Article mis en avant"
- Prévisualisation en temps réel à côté du formulaire
- Validation complète :
 - Titre obligatoire (min 5 caractères)
 - Contenu obligatoire (min 50 caractères)
 - Au moins une catégorie
 - Réseau obligatoire
- Auto-sauvegarde en brouillon toutes les 30 secondes
- Indication visuelle des modifications non sauvegardées

4. Gestion des catégories

- Liste avec couleurs visuelles
- Création/édition/suppression
- Compteur d'articles par catégorie
- Impossible de supprimer une catégorie utilisée

5. Module de notifications

- Formulaire d'envoi de notification pour un article :
 - Liste de destinataires (emails séparés par virgules)
 - Sujet personnalisable (pré-rempli avec le titre)
 - Prévisualisation de l'email HTML généré
- Historique des notifications envoyées avec :
 - Article concerné
 - Nombre de destinataires
 - Date d'envoi
 - Statut (envoyé/échoué)

6. Import de données

- Interface d'upload de fichier JSON
- Affichage du résultat de l'import (succès/erreurs)
- Format JSON attendu :

```
[
{
  "title": "Article importé",
  "content": "Contenu de l'article...",
  "excerpt": "Résumé",
  "author": "Auteur",
  "category": "nom-categorie",
  "network": "nom-reseau"
}
]
```



Contraintes techniques

Obligatoire

- Node.js avec Express ou Fastify
- React (Create React App, Vite ou Next.js)
- Stockage en mémoire OU fichier JSON OU base SQLite (au choix)
- Architecture propre (controllers, services, routes côté back)
- Composants réutilisables côté front
- Gestion complète des erreurs (front + back)
- Validation des données (côté serveur obligatoire)
- Au moins 3 catégories et 2 réseaux pré-chargés au démarrage
- Au moins 10 articles de démo au démarrage

Fortement apprécié

- TypeScript (front et/ou back)
- State management (Context API, Zustand, Redux...)
- Bibliothèque de validation (Zod, Yup, Joi...)
- Tests unitaires (au moins 3-4 tests significatifs)
- Design system ou bibliothèque UI (Material-UI, Chakra, shadcn/ui...)
- Génération d'email HTML avec template réutilisable
- Code splitting et lazy loading
- Accessibilité (navigation clavier, ARIA labels)
- Responsive design

Architecture attendue

Back-end

```
src/  
  controllers/  
  services/  
  routes/  
  models/  
  middlewares/  
  utils/  
  data/ (si fichier JSON)
```

Front-end

```
src/  
  components/
```

common/
articles/
categories/
notifications/
pages/
hooks/
services/ (API calls)
context/ (si state management)
utils/
types/ (si TypeScript)

Critères d'évaluation

Architecture et organisation (25%)

- Structure du projet cohérente et scalable
- Séparation des responsabilités (MVC, services...)
- Réutilisabilité du code
- Choix techniques justifiés

Qualité du code (25%)

- Lisibilité et maintenabilité
- Gestion des erreurs et cas limites
- Validation des données
- Bonnes pratiques JavaScript/TypeScript
- Performance et optimisations

Fonctionnalités (25%)

- Complétude des fonctionnalités demandées
- Robustesse de l'application
- Gestion des états (loading, erreur, succès)
- Cohérence des workflows

UX/UI et professionnalisme (15%)

- Interface intuitive et ergonomique
- Feedback utilisateur systématique
- Cohérence visuelle
- Responsive design
- Accessibilité

Tests et documentation (10%)

- Tests pertinents (si présents)
- README complet et clair
- Documentation du code (si nécessaire)
- Commentaires sur les choix complexes



Livrables attendus

1. Code source

- Repository Git avec commits réguliers et messages clairs
- .gitignore approprié
- Variables d'environnement documentées (.env.example)

2. Documentation (README.md)

Structure attendue :

```
# Titre du projet

## Description
Brève présentation

## Prérequis
- Node.js version X
- npm/yarn

## Installation
```bash
Commandes exactes
```

## Lancement
```bash
Commandes pour démarrer le back et le front
```

## Choix techniques
- Architecture choisie et pourquoi
- Technologies utilisées et justification
- Compromis effectués (si limites de temps)

## Fonctionnalités implémentées
Liste avec statut (✅ complet, ⚠️ partiel, ❌ non fait)

## Ce qui aurait été fait avec plus de temps
- Liste priorisée d'améliorations

## Tests
Comment lancer les tests (si présents)

## Difficultés rencontrées
Points bloquants et solutions trouvées
```

3. Données de démonstration

- Fichier d'import JSON avec 5-10 articles exemples
- Catégories et réseaux pré-configurés



- Au moins 2-3 notifications d'exemple dans l'historique



Instructions de soumission

- 1.** Créez un repository Git public (GitHub, GitLab, Bitbucket)
- 2.** Committez régulièrement avec des messages explicites
- 3.** Assurez-vous que le README contient toutes les instructions
- 4.** Testez l'installation from scratch avant envoi
- 5.** Envoyez le lien du repository à romain@taramgroup.com



Conseils pour réussir

Gestion du temps (4h)

- 0h-0h30 : Setup projet, structure, architecture
- 0h30-2h : Développement back-end + données de démo
- 2h-3h30 : Développement front-end (features principales)
- 3h30-3h50 : Tests, corrections bugs
- 3h50-4h : Documentation, polish final

Priorisation

- 1. Must have :** CRUD articles, catégories, filtres basiques, dashboard simple
- 2. Should have :** Notifications, import, prévisualisation, filtres avancés
- 3. Nice to have :** Tests, TypeScript, design avancé, auto-save

Stratégie recommandée

- Commencez par un MVP fonctionnel bout en bout
- Ajoutez progressivement les features avancées
- Privilégiez la robustesse à la quantité
- Documentez au fur et à mesure
- Testez régulièrement votre application

Ce que nous valorisons

- Pragmatisme : faire des choix adaptés au temps imparti
- Professionnalisme : code production-ready même simple
- Communication : expliquer vos choix et compromis
- Rigueur : validation, gestion erreurs, UX
- Vision : montrer que vous pensez scalabilité et maintenabilité

Bon courage ! Nous avons hâte de découvrir votre travail.